MongoDB 8.0 · SEGURIDAD PRACTICAS

Edición contrastada con los scripts de GitHub Actions. Las cifras de explain() son medidas de una ejecución concreta, no compromisos de rendimiento. Las actividades no cubiertas por el CI se identifican como ampliación hasta tener pruebas individuales.

Seguridad

Cuaderno de laboratorios

Fichas con órdenes, pruebas negativas, explicaciones causales y espacio para resultados.

Entorno canónico: el repositorio GitHub utiliza `compose.yaml` en la raíz y los archivos de `lab/`. **Solo se consideran automáticamente verificados los casos que ejecuta `.github/workflows/laboratorios.yml`.** No todos los experimentos opcionales de estas fichas están automatizados.

Antes del primer laboratorio

Mapa de ejecución sin improvisaciones

Equipamiento y entorno

- Equipo de 8 GB RAM como referencia, Docker Desktop activo, terminal PowerShell o Bash, espacio libre suficiente para los volúmenes y conexión para descargar mongo:8.0.
- MongoDB Community 8.0 y mongosh vienen dentro de las imágenes; Compass es opcional. No se exige licencia Enterprise para los ejercicios esenciales.

Preparación del formador

```
Copy-Item .env.example .env
docker compose up -d
docker compose ps
# después ejecutar /lab/08_configurar_seguridad.js con root
```

Dos contextos distintos

Las pruebas de conexión dependen de usuario, base de autenticación y acción. Es obligatorio preparar el documento de ejemplo con root antes de ensayar accesos anónimos.

Normas de reproducibilidad

Los scripts del paquete contienen datos de laboratorio exclusivamente. Las operaciones que borran ventas o colecciones demo están identificadas. Al repetir una carga, se pierden los índices secundarios de la colección reinicializada. No ejecutarlos sobre servidores externos ni entornos con datos propios.

PREPARAR → EJECUTAR → OBSERVAR → EXPLICAR

Ruta esencial de tres horas

Indexación: carga, línea base, índice simple, comparación, compuesto ESR, text y TTL. Seguridad: conexión anónima rechazada, login SCRAM, cuenta read, denegación insert y explicación TLS/X.509/LDAP. Los demás laboratorios forman itinerario de ampliación, no se prometen todos dentro de los 180 minutos.

> Si el formador no tiene Docker listo previamente, el tiempo de descarga y carga de datos puede consumir la sesión. Ejecutar un ensayo completo en el equipo y conservar la salida antes de impartir.

Laboratorio 01 · Inventario de infraestructura

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Comprobar dos instancias y los puertos publicados.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose ps
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Comprobar dos instancias y los puertos publicados. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La red del contenedor no implica publicar MongoDB en toda la red.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 01 · Inventario de infraestructura

Solución comentada y ampliación

Solución razonada

Comienza por comprobar la IP publicada. mongo-indices no exige credenciales pero su puerto está limitado a localhost; mongo-seguridad exige login para consultar datos protegidos. Especifica cuál de estas propiedades has medido y cuál solo has inspeccionado en configuración.

Por qué ocurre

bindIp controla las interfaces en que escucha mongod. En Docker, publicar 127.0.0.1:27018:27017 expone el servicio únicamente en el loopback del host, pero los contenedores en la red de Compose siguen teniendo otra conectividad interna. La restricción reduce superficie de red; no reemplaza la autenticación. Evita publicar la instancia libre en todas las interfaces de un equipo de aula o de una red Wi-Fi compartida.

Qué no permite concluir

La red del contenedor no implica publicar MongoDB en toda la red. Los puertos publicados empiezan por 127.0.0.1 en Compose; cambiarlos por 0.0.0.0 ampliaría superficie de exposición.

Contraste resuelto con un segundo escenario

```
docker compose ps
```

Comprueba que la publicación es 127.0.0.1:27018 y no 0.0.0.0:27018. Esta revisión no impide que otro proceso local intente conectarse.

Ejercicio de transferencia

Escribir un mapa de superficies expuestas.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar dos instancias y los puertos publicados. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 02 · Primer arranque con credenciales

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Comprobar creación de contenedores y volúmenes.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose up -d
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Comprobar creación de contenedores y volúmenes. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** La contraseña root solo se inicializa cuando el volumen es nuevo.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 02 · Primer arranque con credenciales

Solución comentada y ampliación

Solución razonada

Las variables de la imagen oficial solo se aplican al primer arranque de la base con volumen vacío. Una edición de .env no cambia la contraseña de una cuenta existente. Comprueba logs sin imprimir secretos y evita borrar volúmenes para «probar otra vez» sin reconocer que destruirías datos.

Por qué ocurre

Activar authorization hace que MongoDB compruebe permisos. El contenedor oficial puede crear un administrador inicial al arrancar por primera vez con las variables MONGO_INITDB_ROOT_*, siempre que el directorio de datos esté vacío. Cambiar el archivo .env después no reinicializa cuentas dentro de un volumen existente. Es una fuente habitual de errores en clase. No se debe sugerir borrar volúmenes de forma general: solo los volúmenes nombrados del laboratorio y con aviso de pérdida de datos.

Qué no permite concluir

La contraseña root solo se inicializa cuando el volumen es nuevo. Las variables MONGO_INITDB_ROOT_* solo crean la identidad inicial en un directorio de datos nuevo; cambiar variables no rota cuentas existentes.

Contraste resuelto con un segundo escenario

```
docker compose logs mongo-seguridad --tail 30
```

Los mensajes de inicialización permiten comprobar que el proceso arrancó, pero no sustituyen una lectura real bajo la cuenta creada.

Ejercicio de transferencia

Explicar qué ocurre al cambiar .env con datos ya existentes.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar creación de contenedores y volúmenes. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 03 · Prueba anónima de conexión

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Distinguir shell iniciado de consulta autorizada.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -T mongo-seguridad mongosh --quiet --eval 'db.getSiblingDB("curso_seguridad").secretos.findOne()'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Distinguir shell iniciado de consulta autorizada. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Una base nueva/vacía puede tener matices: confirmar con colección creada por administrador.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 03 · Prueba anónima de conexión

Solución comentada y ampliación

Solución razonada

Abrir un shell no es lo mismo que tener permiso para leer. Debe existir previamente un documento protegido creado por administrador; por eso el preflight de seguridad se ejecuta primero. La lectura anónima debe fallar y el fallo esperado necesita identificar una denegación de autorización, no un puerto incorrecto.

Por qué ocurre

La autenticación responde «¿quién eres?» y la autorización «¿puedes ejecutar esta operación?». Es posible que el cliente abra una conexión TCP y vea el shell, pero falle después al ejecutar find. También es posible que usuario y contraseña sean correctos y que insertOne sea rechazado con Unauthorized. Son dos evidencias diferentes: éxito del inicio de sesión y comprobación de los privilegios. Separarlas evita diagnosticar un error de rol como si fuera una contraseña incorrecta.

Qué no permite concluir

Una base nueva/vacía puede tener matices: confirmar con colección creada por administrador. La operación sin login debe fallar al consultar la colección protegida; una conexión establecida solo prueba la capa de transporte.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

La misma lectura será autorizada para lectorlab y denegada para la sesión anónima. La conexión TCP puede haberse establecido en ambos casos.

Ejercicio de transferencia

Registrar mensaje de error y su causa.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar distinguir shell iniciado de consulta autorizada. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 04 · Autenticación root de formación

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Comprobar authenticatedUsers y authInfo.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -T mongo-seguridad sh -lc 'mongosh --quiet -u adminlab -p "$MONGO_INITDB_ROOT_PASSWORD" --
authenticationDatabase admin --eval "db.runCommand({connectionStatus:1})"'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Comprobar authenticatedUsers y authInfo. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Estas credenciales son exclusivamente ficticias y deben cambiarse en uso real.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 04 · Autenticación root de formación

Solución comentada y ampliación

Solución razonada

El login root acredita que una cuenta creada en admin puede operar en el servidor. No emplees esa cuenta en aplicaciones ni en el laboratorio de denegaciones; si root escribe, es el resultado esperado para un administrador, no un fallo del control de acceso.

Por qué ocurre

En MongoDB, el nombre de usuario está ligado a la base donde se creó. adminlab se autentica en admin; lectorlab, creado en curso_seguridad, lo hace con authSource=curso_seguridad. Probar una clave correcta con authSource equivocado puede fallar; eso no significa que la cuenta no exista. Los roles concedidos pueden referirse a bases diferentes de la que almacena la identidad. En una URI, authSource identifica el origen de credenciales y no es un cambio de autorización.

Qué no permite concluir

Estas credenciales son exclusivamente ficticias y deben cambiarse en uso real. authSource=curso_seguridad es correcto para lectorlab; con admin, aun con clave válida, no coincide la identidad esperada.

Contraste resuelto con un segundo escenario

```
db.runCommand({connectionStatus:1})
```

La información de autenticación efectiva puede ser distinta a la base de datos seleccionada con use. Es necesario conocer la base donde se registró el usuario.

Ejercicio de transferencia

Probar login sin -p para no dejar contraseña en historial.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar authenticatedUsers y authInfo. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 05 · Crear colección controlada

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Comprobar usuario lectorlab y documento ficticio creado.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -T mongo-seguridad sh -lc 'mongosh --quiet -u adminlab -p "$MONGO_INITDB_ROOT_PASSWORD" --
authenticationDatabase admin --file /lab/08_configurar_seguridad.js'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Comprobar usuario lectorlab y documento ficticio creado. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El script es idempotente para el usuario; no sobrescribe su contraseña si existe.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 05 · Crear colección controlada

Solución comentada y ampliación

Solución razonada

La colección secretos almacena únicamente un ejemplo didáctico. Crea el registro mediante root y haz una consulta como root. Su existencia servirá después para las pruebas de lectura positiva y escritura negativa del usuario lectorlab.

Por qué ocurre

La autenticación responde «¿quién eres?» y la autorización «¿puedes ejecutar esta operación?». Es posible que el cliente abra una conexión TCP y vea el shell, pero falle después al ejecutar find. También es posible que usuario y contraseña sean correctos y que insertOne sea rechazado con Unauthorized. Son dos evidencias diferentes: éxito del inicio de sesión y comprobación de los privilegios. Separarlas evita diagnosticar un error de rol como si fuera una contraseña incorrecta.

Qué no permite concluir

El script es idempotente para el usuario; no sobrescribe su contraseña si existe. La operación sin login debe fallar al consultar la colección protegida; una conexión establecida solo prueba la capa de transporte.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

La misma lectura será autorizada para lectorlab y denegada para la sesión anónima. La conexión TCP puede haberse establecido en ambos casos.

Ejercicio de transferencia

Registrar authSource y roles.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar usuario lectorlab y documento ficticio creado. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 06 · Login de solo lectura

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Lectura permitida, escritura denegada como resultado esperado.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -T mongo-seguridad sh -lc 'mongosh --quiet -u lectorlab -p "$LAB_READER_PASSWORD" --
authenticationDatabase curso_seguridad --file /lab/09_comprobar_readonly.js'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Lectura permitida, escritura denegada como resultado esperado. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No confundir Unauthorized con que falló la práctica.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 06 · Login de solo lectura

Solución comentada y ampliación

Solución razonada

La cuenta lectorlab fue creada dentro de curso_seguridad y tiene el rol read. Con ella findOne debe funcionar. Este paso positivo es imprescindible: si falla, los posteriores errores de insert pueden proceder de una mala autenticación.

Por qué ocurre

read da acceso a operaciones de consulta en la base indicada; readWrite añade operaciones de modificación de datos. Los roles administrativos permiten acciones mucho más amplias. Crear un usuario único para todas las tareas del curso sería cómodo pero impediría comprobar la separación de privilegios. Documenta roles y bases de datos, no una clasificación moral del usuario: el mismo usuario puede tener distintos privilegios en distintas bases.

Qué no permite concluir

No confundir Unauthorized con que falló la práctica. read permite find y no insert, update ni delete; readWrite expande permisos dentro de su base de datos designada.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUser("lectorlab")
```

El administrador debe observar que el rol está limitado a read sobre la base de formación. Una cuenta admin no es necesaria para la aplicación lectora.

Ejercicio de transferencia

Repetir consulta desde Compass.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar lectura permitida, escritura denegada como resultado esperado. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 07 · Base de autenticación equivocada

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

La autenticación debe fallar porque lectorlab fue creado en curso_seguridad.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -it mongo-seguridad mongosh --quiet -u lectorlab --authenticationDatabase admin --eval
'db.runCommand({connectionStatus:1})'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

La autenticación debe fallar porque lectorlab fue creado en curso_seguridad. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Confirmar la base del usuario antes de modificar contraseñas.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 07 · Base de autenticación equivocada

Solución comentada y ampliación

Solución razonada

El argumento --authenticationDatabase debe señalar curso_seguridad para lectorlab, no admin. Si se usa admin, la misma contraseña no identifica la cuenta registrada donde se espera. Conserva el mensaje concreto y vuelve a probar la URI corregida.

Por qué ocurre

En MongoDB, el nombre de usuario está ligado a la base donde se creó. adminlab se autentica en admin; lectorlab, creado en curso_seguridad, lo hace con authSource=curso_seguridad. Probar una clave correcta con authSource equivocado puede fallar; eso no significa que la cuenta no exista. Los roles concedidos pueden referirse a bases diferentes de la que almacena la identidad. En una URI, authSource identifica el origen de credenciales y no es un cambio de autorización.

Qué no permite concluir

Confirmar la base del usuario antes de modificar contraseñas. authSource=curso_seguridad es correcto para lectorlab; con admin, aun con clave válida, no coincide la identidad esperada.

Contraste resuelto con un segundo escenario

```
db.runCommand({connectionStatus:1})
```

La información de autenticación efectiva puede ser distinta a la base de datos seleccionada con use. Es necesario conocer la base donde se registró el usuario.

Ejercicio de transferencia

Representar el proceso de selección de authSource.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar la autenticación debe fallar porque lectorlab fue creado en curso_seguridad. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 08 · Contraseña incorrecta

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Error Authentication failed esperado.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose exec -it mongo-seguridad mongosh --quiet -u lectorlab -p "incorrecta" --authenticationDatabase
curso_seguridad --eval 'db.runCommand({connectionStatus:1})'
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Error Authentication failed esperado. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No usar cuentas personales en pruebas negativas.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 08 · Contraseña incorrecta

Solución comentada y ampliación

Solución razonada

Una contraseña incorrecta provoca error de autenticación. Evita confundirlo con Unauthorized: la sesión nunca llegó a superar la verificación de identidad. No imprimas ni guardes una contraseña real al documentar el test; todos los secretos de esta guía son valores sintéticos de aula.

Por qué ocurre

Authentication failed significa que la credencial presentada no valida la identidad esperada; Unauthorized aparece cuando una identidad ya aceptada intenta una operación no permitida. Además puede haber fallos de conexión, resolución del nombre, TLS o base de autenticación. La secuencia de diagnóstico no empieza por cambiar contraseñas: comprueba primero puerto y servidor, luego mecanismo y authSource, y por último roles. Captura el error esperado sin incrustar secretos completos en el informe.

Qué no permite concluir

No usar cuentas personales en pruebas negativas. Authentication failed señala verificación de credencial; Unauthorized señala que la operación carece de privilegio después de la conexión.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.findOne()
```

La prueba positiva debe preceder al error de escritura. Si falla find, no se ha demostrado una política read; existe un problema previo.

Ejercicio de transferencia

Distinguir de error TCP Connection refused.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar error Authentication failed esperado. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 09 · Escritura prohibida al lector

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Se espera Unauthorized; comprobar como root que no se insertó.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u lectorlab --authenticationDatabase curso_seguridad
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({dato:"NO-DEBE-GUARDARSE"})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Se espera Unauthorized; comprobar como root que no se insertó. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Una aplicación debe tratar correctamente la operación denegada.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 09 · Escritura prohibida al lector

Solución comentada y ampliación

Solución razonada

Si find funciona e insert falla con código 13, el control de mínimo privilegio ha quedado evidenciado. Repite countDocuments para comprobar que no se escribió nada. El error esperado no debe considerarse fallo del laboratorio: precisamente evita una modificación sin autorización.

Por qué ocurre

El mínimo privilegio se prueba con una cuenta que puede leer secretos_demo y no puede insertar en ella. Si ambos comandos funcionan, el laboratorio ha concedido demasiado. Si ambos fallan, no se ha demostrado una política de solo lectura, sino un problema de autenticación o contexto. La prueba positiva es tan importante como la negativa: tiene que existir una operación autorizada que confirme que la sesión realmente se autenticó.

Qué no permite concluir

Una aplicación debe tratar correctamente la operación denegada. La denegación de insert a lectorlab verifica autorización del lado servidor, no una restricción de interfaz o formulario.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"no"})
```

Esta inserción, realizada por un usuario read, debe fallar. Verifica con una lectura que no hubo alteración de datos.

Ejercicio de transferencia

Explicar qué permisos específicos faltan.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar se espera Unauthorized; comprobar como root que no se insertó. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 10 · Update prohibido al lector

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Se espera Unauthorized y la descripción original permanece.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u lectorlab --authenticationDatabase curso_seguridad
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").secretos.updateOne({codigo:"SECRETO-DEMO"},{$set:{descripcion:"alterado"}})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Se espera Unauthorized y la descripción original permanece. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No probar modificaciones sobre datos reales de alumnos.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 10 · Update prohibido al lector

Solución comentada y ampliación

Solución razonada

Una cuenta read también debe carecer de permisos para updateOne. La operación negativa no debe alterar el documento. Verifica después con una lectura, que sí es permitida, que la descripción conserva el valor original.

Por qué ocurre

El mínimo privilegio se prueba con una cuenta que puede leer secretos_demo y no puede insertar en ella. Si ambos comandos funcionan, el laboratorio ha concedido demasiado. Si ambos fallan, no se ha demostrado una política de solo lectura, sino un problema de autenticación o contexto. La prueba positiva es tan importante como la negativa: tiene que existir una operación autorizada que confirme que la sesión realmente se autenticó.

Qué no permite concluir

No probar modificaciones sobre datos reales de alumnos. La denegación de insert a lectorlab verifica autorización del lado servidor, no una restricción de interfaz o formulario.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"no"})
```

Esta inserción, realizada por un usuario read, debe fallar. Verifica con una lectura que no hubo alteración de datos.

Ejercicio de transferencia

Comparar operaciones read y update.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar se espera Unauthorized y la descripción original permanece. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 11 · Delete prohibido al lector

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Se espera Unauthorized y documento intacto.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u lectorlab --authenticationDatabase curso_seguridad
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").secretos.deleteOne({codigo:"SECRETO-DEMO"})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Se espera Unauthorized y documento intacto. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Fallo de autorización no equivale a ausencia de documento.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 11 · Delete prohibido al lector

Solución comentada y ampliación

Solución razonada

Una operación deleteOne denegada confirma que read no permite eliminar. El documento debe continuar presente. Enseña al alumno a separar denegación por rol de intento sobre una colección inexistente.

Por qué ocurre

El mínimo privilegio se prueba con una cuenta que puede leer secretos_demo y no puede insertar en ella. Si ambos comandos funcionan, el laboratorio ha concedido demasiado. Si ambos fallan, no se ha demostrado una política de solo lectura, sino un problema de autenticación o contexto. La prueba positiva es tan importante como la negativa: tiene que existir una operación autorizada que confirme que la sesión realmente se autenticó.

Qué no permite concluir

Fallo de autorización no equivale a ausencia de documento. La denegación de insert a lectorlab verifica autorización del lado servidor, no una restricción de interfaz o formulario.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"no"})
```

Esta inserción, realizada por un usuario read, debe fallar. Verifica con una lectura que no hubo alteración de datos.

Ejercicio de transferencia

Definir un test automatizable del límite de permisos.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar se espera Unauthorized y documento intacto. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 12 · Lista de usuarios con root

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Comprobar existencia y roles de lectorlab.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Comprobar existencia y roles de lectorlab. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Usuario no debe consultar el inventario administrativo sin privilegio.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 12 · Lista de usuarios con root

Solución comentada y ampliación

Solución razonada

getUsers y getUser son operaciones para cuenta con privilegios de administración. El alumno de solo lectura no necesita inventariar usuarios del sistema. Construye una tabla con usuario, base de creación y roles; no imprimas hashes de contraseña ni información innecesaria.

Por qué ocurre

Separar la cuenta de administración de la cuenta de aplicación permite atribuir acciones, rotar secretos y limitar daños. El operador que crea roles no debería prestar sus credenciales a un programa de consulta. En producción se aplican políticas de identidades, secretos y auditoría; en clase basta con crear usuarios diferenciados y registrar para cada uno qué acciones están habilitadas. Aumentar privilegios solo para salvar un error de laboratorio elimina precisamente la lección que se intenta impartir.

Qué no permite concluir

Usuario no debe consultar el inventario administrativo sin privilegio. Dos cuentas con privilegios similares siguen permitiendo rotación y atribución separadas si no comparten identidades.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Un inventario de identidades permite separar cuentas humanas, de aplicación y de auditoría, cada una con permisos diferentes.

Ejercicio de transferencia

Explicar por qué root no debe usarse por aplicaciones.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar existencia y roles de lectorlab. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 13 · Crear cuenta de auditoría

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Verificar cuenta adicional de solo lectura.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").createUser({user:"auditor_lab",pwd:passwordPrompt(),roles:
[{role:"read",db:"curso_seguridad"}]})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Verificar cuenta adicional de solo lectura. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No ejecutar createUser dos veces sin verificar si el usuario existe.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 13 · Crear cuenta de auditoría

Solución comentada y ampliación

Solución razonada

Una identidad de auditoría limitada puede investigar sin capacidad de modificar datos. La definición exacta del rol depende de qué operaciones debas consultar. Evita usar root solo porque falta permiso: analiza cuál es la acción concreta que necesita la tarea.

Por qué ocurre

Separar la cuenta de administración de la cuenta de aplicación permite atribuir acciones, rotar secretos y limitar daños. El operador que crea roles no debería prestar sus credenciales a un programa de consulta. En producción se aplican políticas de identidades, secretos y auditoría; en clase basta con crear usuarios diferenciados y registrar para cada uno qué acciones están habilitadas. Aumentar privilegios solo para salvar un error de laboratorio elimina precisamente la lección que se intenta impartir.

Qué no permite concluir

No ejecutar createUser dos veces sin verificar si el usuario existe. Dos cuentas con privilegios similares siguen permitiendo rotación y atribución separadas si no comparten identidades.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Un inventario de identidades permite separar cuentas humanas, de aplicación y de auditoría, cada una con permisos diferentes.

Ejercicio de transferencia

Crear cuenta de servicio con alcance más limitado.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar verificar cuenta adicional de solo lectura. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 14 · Comprobar un rol real

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Leer privilegios efectivos según respuesta del servidor.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").getUser("lectorlab",{showPrivileges:true})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Leer privilegios efectivos según respuesta del servidor. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No deducir privilegios solo del nombre de la cuenta.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 14 · Comprobar un rol real

Solución comentada y ampliación

Solución razonada

La respuesta de connectionStatus o getUser permite explicar quién es la identidad actual y qué roles se han reconocido. La comprobación tiene sentido si se relaciona un rol con una operación real. Un nombre de usuario en una captura, por sí solo, no prueba que exista acceso de lectura o escritura.

Por qué ocurre

read da acceso a operaciones de consulta en la base indicada; readWrite añade operaciones de modificación de datos. Los roles administrativos permiten acciones mucho más amplias. Crear un usuario único para todas las tareas del curso sería cómodo pero impediría comprobar la separación de privilegios. Documenta roles y bases de datos, no una clasificación moral del usuario: el mismo usuario puede tener distintos privilegios en distintas bases.

Qué no permite concluir

No deducir privilegios solo del nombre de la cuenta. read permite find y no insert, update ni delete; readWrite expande permisos dentro de su base de datos designada.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUser("lectorlab")
```

El administrador debe observar que el rol está limitado a read sobre la base de formación. Una cuenta admin no es necesaria para la aplicación lectora.

Ejercicio de transferencia

Comparar rol read frente a readWrite.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar leer privilegios efectivos según respuesta del servidor. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 15 · Crear usuario de escritura

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Verificar que puede escribir en curso_seguridad, no en admin.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").createUser({user:"editor_practica",pwd:passwordPrompt(),roles:
[{role:"readWrite",db:"curso_seguridad"}]})
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Verificar que puede escribir en curso_seguridad, no en admin. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Credenciales ficticias; borrar o rotar al terminar.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 15 · Crear usuario de escritura

Solución comentada y ampliación

Solución razonada

Crear readWrite permite comparar dos cuentas sobre la misma colección. Prueba un insert bajo la nueva identidad en una colección demo separada. No confundas el éxito de readWrite con permiso administrativo sobre la base admin.

Por qué ocurre

read da acceso a operaciones de consulta en la base indicada; readWrite añade operaciones de modificación de datos. Los roles administrativos permiten acciones mucho más amplias. Crear un usuario único para todas las tareas del curso sería cómodo pero impediría comprobar la separación de privilegios. Documenta roles y bases de datos, no una clasificación moral del usuario: el mismo usuario puede tener distintos privilegios en distintas bases.

Qué no permite concluir

Credenciales ficticias; borrar o rotar al terminar. read permite find y no insert, update ni delete; readWrite expande permisos dentro de su base de datos designada.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUser("lectorlab")
```

El administrador debe observar que el rol está limitado a read sobre la base de formación. Una cuenta admin no es necesaria para la aplicación lectora.

Ejercicio de transferencia

Probar acceso a otra base de datos y explicar aislamiento.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar verificar que puede escribir en curso_seguridad, no en admin. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 16 · Retirar permisos temporalmente

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Tras retirar rol, editor deja de escribir.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").revokeRolesFromUser("editor_practica",[{role:"readWrite",db:"curso_seguridad"}])
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Tras retirar rol, editor deja de escribir. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Evitar cortar el acceso de root durante ejercicios.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 16 · Retirar permisos temporalmente

Solución comentada y ampliación

Solución razonada

Revocar un rol afecta a las capacidades asociadas a esa identidad; cuando se ensayan cambios, conviene abrir una sesión nueva para evitar atribuir un resultado a cachés de autorización. Registra primero el rol anterior y conserva un comando de restauración.

Por qué ocurre

El mínimo privilegio se prueba con una cuenta que puede leer secretos_demo y no puede insertar en ella. Si ambos comandos funcionan, el laboratorio ha concedido demasiado. Si ambos fallan, no se ha demostrado una política de solo lectura, sino un problema de autenticación o contexto. La prueba positiva es tan importante como la negativa: tiene que existir una operación autorizada que confirme que la sesión realmente se autenticó.

Qué no permite concluir

Evitar cortar el acceso de root durante ejercicios. La denegación de insert a lectorlab verifica autorización del lado servidor, no una restricción de interfaz o formulario.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").secretos.insertOne({prueba:"no"})
```

Esta inserción, realizada por un usuario read, debe fallar. Verifica con una lectura que no hubo alteración de datos.

Ejercicio de transferencia

Restituir con grantRolesToUser para cerrar el estado.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar tras retirar rol, editor deja de escribir. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 17 · Recuperar privilegios

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Repetir operación permitida y registrar la diferencia.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u adminlab --authenticationDatabase admin
```

Paso 2 · Ejecutar dentro del shell

```
db.getSiblingDB("curso_seguridad").grantRolesToUser("editor_practica",[{role:"readWrite",db:"curso_seguridad"}])
```

Precondición que cambia la interpretación

El usuario lectorlab se crea con /lab/08_configurar_seguridad.js usando adminlab en admin. Si el ejercicio necesita esa cuenta, preparar primero este script y comprobar su resultado.

Evidencia concreta a registrar

Repetir operación permitida y registrar la diferencia. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Autorización se verifica con operaciones, no solo con getUser.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 17 · Recuperar privilegios

Solución comentada y ampliación

Solución razonada

Tras restaurar el rol se debe confirmar tanto el listado de roles como una consulta positiva. Las pruebas reversibles son preferibles a borrar usuarios o datos. Una política de aula debe finalizar con el mismo acceso que tenía antes del experimento.

Por qué ocurre

Separar la cuenta de administración de la cuenta de aplicación permite atribuir acciones, rotar secretos y limitar daños. El operador que crea roles no debería prestar sus credenciales a un programa de consulta. En producción se aplican políticas de identidades, secretos y auditoría; en clase basta con crear usuarios diferenciados y registrar para cada uno qué acciones están habilitadas. Aumentar privilegios solo para salvar un error de laboratorio elimina precisamente la lección que se intenta impartir.

Qué no permite concluir

Autorización se verifica con operaciones, no solo con getUser. Dos cuentas con privilegios similares siguen permitiendo rotación y atribución separadas si no comparten identidades.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("curso_seguridad").getUsers()
```

Un inventario de identidades permite separar cuentas humanas, de aplicación y de auditoría, cada una con permisos diferentes.

Ejercicio de transferencia

Distinguir cambio de permiso de rotación de contraseña.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar repetir operación permitida y registrar la diferencia. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 18 · Describir estructura de conexión

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Identificar host, puerto, base objetivo, usuario y authSource.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
mongodb://lectorlab@127.0.0.1:27018/curso_seguridad?authSource=curso_seguridad
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Identificar host, puerto, base objetivo, usuario y authSource. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** El texto con <password> es un esquema, no un comando de ejecución.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 18 · Describir estructura de conexión

Solución comentada y ampliación

Solución razonada

Descompón una URI de MongoDB en host, puerto, base por defecto, usuario y authSource. El prefijo mongodb:// no significa cifrado. Una cadena puede autenticar correctamente sin TLS y seguir sin proteger el transporte; esta distinción motiva el bloque de certificados.

Por qué ocurre

Una conexión segura necesita llegar al servidor correcto, identificarlo, usar un canal cifrado y limitar las operaciones del usuario. Una URI con usuario y contraseña pero sin TLS puede autenticar sin proteger el tránsito. Un canal TLS con usuario root protege el transporte pero deja demasiado poder a la aplicación. En la guía se descompone la solución en conexión, canal, identidad y autorización; cada pieza tiene una prueba distinta.

Qué no permite concluir

El texto con <password> es un esquema, no un comando de ejecución. La cadena URI puede filtrar contraseñas por shell o capturas; usar interfaz de entrada protegida en lugar de incorporarlas a documentos.

Contraste resuelto con un segundo escenario

```
docker compose exec -it mongo-seguridad mongosh --host localhost --port 27017
```

Llegar al servidor sin credenciales no equivale a autenticar. Un shell iniciado no debe confundir al alumno cuando find requiere permisos.

Ejercicio de transferencia

Explicar URL-encoding de caracteres reservados.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar identificar host, puerto, base objetivo, usuario y authSource. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 19 · Probar desde MongoDB Compass

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Validar conexión con autenticación configurada por campos en Compass.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
// URI de referencia: mongodb://127.0.0.1:27018/curso_seguridad?authSource=curso_seguridad
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Validar conexión con autenticación configurada por campos en Compass. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No mostrar contraseña en el proyector ni usar root.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 19 · Probar desde MongoDB Compass

Solución comentada y ampliación

Solución razonada

En Compass prueba la misma cuenta read y selecciona la base de autenticación correcta. La interfaz no debe llevar al alumno a creer que las restricciones se aplican solo al shell: la autorización reside en el servidor y debe ser idéntica para todos los clientes.

Por qué ocurre

Una conexión segura necesita llegar al servidor correcto, identificarlo, usar un canal cifrado y limitar las operaciones del usuario. Una URI con usuario y contraseña pero sin TLS puede autenticar sin proteger el tránsito. Un canal TLS con usuario root protege el transporte pero deja demasiado poder a la aplicación. En la guía se descompone la solución en conexión, canal, identidad y autorización; cada pieza tiene una prueba distinta.

Qué no permite concluir

No mostrar contraseña en el proyector ni usar root. La cadena URI puede filtrar contraseñas por shell o capturas; usar interfaz de entrada protegida en lugar de incorporarlas a documentos.

Contraste resuelto con un segundo escenario

```
docker compose exec -it mongo-seguridad mongosh --host localhost --port 27017
```

Llegar al servidor sin credenciales no equivale a autenticar. Un shell iniciado no debe confundir al alumno cuando find requiere permisos.

Ejercicio de transferencia

Capturar error genérico sin exponer secretos.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar validar conexión con autenticación configurada por campos en Compass. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 20 · Prueba sin puerto publicado

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Comprobar que la escucha externa no es 0.0.0.0.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
// En compose.yaml inspeccionar:
ports:
  - "127.0.0.1:27018:27017"
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Comprobar que la escucha externa no es 0.0.0.0. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No editar publicación de puertos en red corporativa sin autorización.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 20 · Prueba sin puerto publicado

Solución comentada y ampliación

Solución razonada

Eliminar la publicación del puerto del host no detiene el servicio dentro de la red de Compose. Esta práctica es una comparación de exposición, no de permisos. Antes de modificar Compose, documenta el estado anterior y asegura una reversión clara.

Por qué ocurre

bindIp controla las interfaces en que escucha mongod. En Docker, publicar 127.0.0.1:27018:27017 expone el servicio únicamente en el loopback del host, pero los contenedores en la red de Compose siguen teniendo otra conectividad interna. La restricción reduce superficie de red; no reemplaza la autenticación. Evita publicar la instancia libre en todas las interfaces de un equipo de aula o de una red Wi-Fi compartida.

Qué no permite concluir

No editar publicación de puertos en red corporativa sin autorización. Los puertos publicados empiezan por 127.0.0.1 en Compose; cambiarlos por 0.0.0.0 ampliaría superficie de exposición.

Contraste resuelto con un segundo escenario

```
docker compose ps
```

Comprueba que la publicación es 127.0.0.1:27018 y no 0.0.0.0:27018. Esta revisión no impide que otro proceso local intente conectarse.

Ejercicio de transferencia

Comparar host y red interna de Docker.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar comprobar que la escucha externa no es 0.0.0.0. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 21 · Inspeccionar logs sin datos sensibles

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Observar mensajes del servicio y diferenciar fallos de arranque de autenticación.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Ejecutar desde terminal (PowerShell/Bash)

```
docker compose logs --tail=40 mongo-seguridad
```

Precondición que cambia la interpretación

No se atribuye el éxito de SCRAM a TLS ni se usa una cuenta administradora para probar privilegios de alumno.

Evidencia concreta a registrar

Observar mensajes del servicio y diferenciar fallos de arranque de autenticación. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** No asumir que logs de Community son auditoría completa.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 21 · Inspeccionar logs sin datos sensibles

Solución comentada y ampliación

Solución razonada

Los registros de arranque muestran si el proceso está sano; los errores de permisos suelen producir pistas diagnósticas. No confundas una línea en un log con una auditoría completa. Identifica hora, servicio y error sin pegar contraseñas.

Por qué ocurre

Los logs de Docker y del servidor sirven para investigar arranque, fallos de conexión y eventos visibles; no prueban que exista una auditoría regulatoria completa. Los sistemas con requisitos exigentes necesitan definir qué eventos se registran, quién puede leerlos y cuánto tiempo se conservan. Evita mostrar secretos en los mensajes de diagnóstico. El ejercicio de revisión consiste en distinguir una denegación de acceso de un fallo del contenedor, no en producir miles de líneas sin interpretación.

Qué no permite concluir

No asumir que logs de Community son auditoría completa. docker logs aporta diagnóstico pero no todas las evidencias de auditoría exigibles; las funciones de auditoría empresarial tienen otro alcance.

Contraste resuelto con un segundo escenario

```
docker compose logs mongo-seguridad --tail 30
```

Busca un evento relacionado con el problema observado, no un mensaje aleatorio. El registro de arranque no reemplaza auditoría formal.

Ejercicio de transferencia

Determinar qué eventos registrarías con Enterprise.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar observar mensajes del servicio y diferenciar fallos de arranque de autenticación. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 22 · Certificados: inspección X.509

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Identificar emisor, nombre SAN y vigencia de un certificado de demostración.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Actividad ampliada o de análisis; leer requisitos

```
bash scripts/generar-certificados.sh
openssl x509 -in certs/server.crt -noout -subject -issuer -dates -ext subjectAltName
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Identificar emisor, nombre SAN y vigencia de un certificado de demostración. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Requiere Bash, Git Bash o WSL, OpenSSL y Docker; no utilizar las claves generadas en producción.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 22 · Certificados: inspección X.509

Solución comentada y ampliación

Solución razonada

El certificado de demostración debe mostrar sujeto, emisor, vigencia y SAN. El fichero con la clave privada necesita protección; una captura de openssl x509 -text sobre el certificado público no requiere revelar esa clave. Verifica que la CA y los certificados corresponden al mismo laboratorio.

Por qué ocurre

Una autoridad de certificación firma certificados; un certificado contiene nombre, emisor, vigencia, clave pública y extensiones. El archivo PEM que utiliza el servidor puede combinar certificado y clave privada, por lo que no debe adjuntarse a repositorios ni a material compartido después de generar claves reales. El alumno puede inspeccionar subject, issuer y fechas mediante openssl sin publicar la clave. El certificado cliente necesita también restricciones de uso adecuadas.

Qué no permite concluir

Requiere Bash, Git Bash o WSL, OpenSSL y Docker; no utilizar las claves generadas en producción. El certificado presenta subject, issuer, validez y SAN; firmar un certificado no asigna automáticamente roles en MongoDB.

Contraste resuelto con un segundo escenario

```
openssl x509 -in certs/client.crt -noout -subject -issuer -dates
```

El subject identifica al titular de un certificado; todavía faltan la confianza de la CA y el registro como usuario externo para conceder roles.

Ejercicio de transferencia

Comparar el sujeto y uso extendido del certificado cliente.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar identificar emisor, nombre SAN y vigencia de un certificado de demostración. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 23 · TLS: construir cadena de confianza

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Una conexión TLS validada con la CA permite ping; no autoriza automáticamente la lectura. En GitHub Actions el recorrido completo (incluida la creación del usuario X.509) se ejecuta mediante scripts/probar-seguridad.sh.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Actividad ampliada o de análisis; leer requisitos

```
docker compose up -d --wait mongo-tls
docker compose exec -T mongo-tls mongosh --quiet --tls --tlsCAFile /certs/ca.crt --eval "db.adminCommand({ping:1})"
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Una conexión TLS validada con la CA permite ping; no autoriza automáticamente la lectura. En GitHub Actions el recorrido completo (incluida la creación del usuario X.509) se ejecuta mediante scripts/probar-seguridad.sh. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Generar certificados primero y arrancar mongo-tls con los certificados creados; sin TLS el acceso al tercer servidor debe fallar.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 23 · TLS: construir cadena de confianza

Solución comentada y ampliación

Solución razonada

TLS requireTLS significa que un cliente que no negocia TLS no puede usar el servicio. Una CA firmante permite verificar identidad, pero la validación también incluye SAN y caducidad. Rechazar un certificado es un resultado positivo si el motivo es el previsto.

Por qué ocurre

TLS establece un canal cifrado y autentica al servidor si el cliente valida su certificado y la cadena de confianza. El certificado del servidor debe corresponder al nombre que el cliente utiliza. En un entorno de formación puede construirse una CA local, pero esa CA no es una autoridad confiable para producción. La configuración requireTLS rechaza las conexiones que no negocian TLS; la protección de tránsito no cifra por sí misma los datos persistidos en el volumen.

Qué no permite concluir

Generar certificados primero y arrancar mongo-tls con los certificados creados; sin TLS el acceso al tercer servidor debe fallar. TLS protege comunicación entre puntos; para comprobar identidad del servidor el cliente debe confiar en la CA y validar el nombre.

Contraste resuelto con un segundo escenario

```
openssl x509 -in certs/server.crt -noout -subject -issuer -dates
```

Un certificado válido para el canal debe tener emisor confiable, vigencia y nombre aceptado; proteger el enlace no cifra automáticamente el volumen.

Ejercicio de transferencia

Contrastar la negociación TLS exitosa con un intento de conexión sin --tls.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar una conexión TLS validada con la CA permite ping; no autoriza automáticamente la lectura. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 24 · x509: conectar con identidad cliente

Ejecución paso a paso

Nivel de evidencia: fenómeno principal reproducido y comprobado en GitHub Actions. Los contrastes de ampliación de esta ficha no se deducen automáticamente de ese resultado.

Resultado profesional perseguido

Autenticación $external por certificado; lectura permitida y escritura denegada.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Actividad ampliada o de análisis; leer requisitos

```
docker compose exec -T mongo-tls mongosh --quiet --host localhost --tls --tlsCAFile /certs/ca.crt --
tlsCertificateKeyFile /certs/client.pem --authenticationMechanism MONGODB-X509 --authenticationDatabase '$external' --
file /lab/14_verificar_x509.js
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Autenticación $external por certificado; lectura permitida y escritura denegada. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Exige antes crear el usuario $external con el script 13, desde una cuenta administradora por TLS.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 24 · x509: conectar con identidad cliente

Solución comentada y ampliación

Solución razonada

Para MONGODB-X509 se registra el subject del certificado cliente como usuario en $external. Antes del ejercicio 24 se debe ejecutar el script validado scripts/probar-seguridad.sh que crea las identidades TLS y X.509. Un certificado firmado sin usuario correspondiente no concede read. El ejemplo ampliado documenta la secuencia real, pero requiere arrancar el servicio TLS y generar certificados; no es la práctica esencial de tres horas.

Por qué ocurre

La autenticación de cliente X.509 agrega un paso a TLS: el servidor no solo demuestra su identidad, también el cliente presenta certificado. El subject compatible con RFC2253 debe estar creado como usuario en la base $external con los roles adecuados. Sin esa correspondencia, presentar un certificado firmado no concede permiso de lectura. El laboratorio ampliado debe verificar, en este orden, cadena TLS, reconocimiento del sujeto y autorización del comando find.

Qué no permite concluir

Exige antes crear el usuario $external con el script 13, desde una cuenta administradora por TLS. X.509 autentica un sujeto registrado en $external y asociado a un rol; el certificado no concede derechos solo por ser válido.

Contraste resuelto con un segundo escenario

```
db.getSiblingDB("$external").getUsers()
```

Si se configuró X.509, el subject del cliente debe coincidir con la identidad creada en $external. TLS exitoso sin rol no basta para hacer find.

Ejercicio de transferencia

Intentar un certificado no registrado y explicar por qué no debe recibir roles.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar autenticación $external por certificado; lectura permitida y escritura denegada. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 25 · LDAP: trazado de arquitectura

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Dibujar pasos de identidad, grupo y privilegio.

Dónde ejecutar y con qué preparación

Ejercicio de arquitectura sobre LDAP Enterprise. No se ejecuta contra Community.

Actividad ampliada o de análisis; leer requisitos

```
Diagrama: cliente -> MongoDB Enterprise -> LDAP -> grupo -> rol
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Dibujar pasos de identidad, grupo y privilegio. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** LDAP no puede probarse con Community base; obsoleto desde 8.0.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 25 · LDAP: trazado de arquitectura

Solución comentada y ampliación

Solución razonada

La integración LDAP necesita Enterprise y un directorio real; aquí se dibuja el flujo de identidad y mapeo de grupos. Es correcto evaluar si AnalistasVentas recibe read y OperadoresVentas readWrite, pero no afirmar que se ejecutó una autenticación LDAP sobre Community.

Por qué ocurre

LDAP permite centralizar usuarios y grupos en un directorio corporativo. En MongoDB autogestionado su integración relevante pertenece a Enterprise y está obsoleta desde la versión 8.0. No tiene sentido instalar un servidor LDAP improvisado para simular que Community ofrece una función de Enterprise. El alumnado debe comprender el flujo: cliente, servidor MongoDB, consulta al directorio, mapeo a roles y verificación de una operación. Un diagrama de ese flujo es actividad válida, pero no equivale a integración ejecutada.

Qué no permite concluir

LDAP no puede probarse con Community base; obsoleto desde 8.0. LDAP requiere servidor y edición compatibles; su obsolescencia desde MongoDB 8.0 aconseja valorar alternativas antes de proyectos nuevos.

Contraste resuelto con un segundo escenario

```
Diagrama: Cliente → MongoDB Enterprise → Directorio LDAP → Grupos
```

La integración requiere infraestructura y edición que no están presentes en Community. La actividad comprueba comprensión de arquitectura, no un inicio de sesión LDAP real.

Ejercicio de transferencia

Comparar federación de identidad con cuentas locales.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar dibujar pasos de identidad, grupo y privilegio. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 26 · Cifrado en reposo y responsabilidad

Ejecución paso a paso

Nivel de evidencia: ejercicio de ampliación no ejecutado como comprobación independiente en GitHub Actions. Sus resultados son hipótesis a verificar, no salidas medidas.

Resultado profesional perseguido

Explicar por qué persistencia en Docker no demuestra cifrado.

Dónde ejecutar y con qué preparación

Terminal PowerShell/Bash, salvo los comandos db.* que pertenecen a mongosh.

Actividad ampliada o de análisis; leer requisitos

```
docker volume ls
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Explicar por qué persistencia en Docker no demuestra cifrado. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Cifrado nativo WiredTiger es Enterprise; cifrado de volumen depende del anfitrión.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 26 · Cifrado en reposo y responsabilidad

Solución comentada y ampliación

Solución razonada

No deduzcas que un volumen de Docker está cifrado por estar montado. El cifrado nativo del motor autogestionado es Enterprise; otras capas como cifrado del disco del host pueden proteger determinados riesgos. Enumera quién custodia la clave y cómo se recupera una copia.

Por qué ocurre

El cifrado en reposo trata la lectura no autorizada de ficheros persistidos o copias. MongoDB Enterprise autogestionado admite cifrado nativo del motor WiredTiger; MongoDB Atlas proporciona opciones de cifrado gestionado. Guardar un volumen Docker no activa esa característica en Community. Cifrado de disco del sistema operativo, cifrado de almacenamiento en nube y cifrado nativo del motor son capas diferentes, con responsabilidades y escenarios de recuperación distintos.

Qué no permite concluir

Cifrado nativo WiredTiger es Enterprise; cifrado de volumen depende del anfitrión. Persistir ficheros MongoDB en Docker no los cifra automáticamente; el cifrado nativo del motor WiredTiger es característica Enterprise.

Contraste resuelto con un segundo escenario

```
docker volume ls
```

La presencia de volúmenes muestra persistencia, NO confirma cifrado. El motor WiredTiger cifrado en reposo autogestionado es una función Enterprise.

Ejercicio de transferencia

Diseñar qué claves deben quedar fuera de la máquina.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar explicar por qué persistencia en Docker no demuestra cifrado. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 27 · Auditoría final de seguridad

Ejecución paso a paso

Nivel de evidencia: parte del comportamiento se ha observado en GitHub Actions; la secuencia o interpretación específica de esta ficha necesita ensayo individual antes de impartirla.

Resultado profesional perseguido

Entregar matriz autenticado/no autenticado, read/write y riesgo.

Dónde ejecutar y con qué preparación

Dentro de mongosh conectado al contenedor mongo-seguridad con la cuenta indicada; no en PowerShell.

Paso 1 · Abrir mongosh con la identidad requerida

```
docker compose exec -it mongo-seguridad mongosh -u lectorlab --authenticationDatabase curso_seguridad
```

Paso 2 · Ejecutar dentro del shell

```
db.runCommand({connectionStatus:1})
```

Precondición que cambia la interpretación

Genera certificados docentes antes de la prueba. TLS y X.509 funcionan en MongoDB Community; para LDAP o cifrado nativo en reposo sí se requieren otras capacidades.

Evidencia concreta a registrar

Entregar matriz autenticado/no autenticado, read/write y riesgo. Registra el contador o error concreto observado, la colección y el comando utilizado. No sustituyas una evidencia ejecutada por una predicción.

> **Si no obtienes el resultado:** Una lista de comprobación no reemplaza pruebas de penetración.

Hoja para registrar salida real y conclusión

Qué esperaba	Qué observé	Qué puedo concluir

Laboratorio 27 · Auditoría final de seguridad

Solución comentada y ampliación

Solución razonada

La auditoría final pide tres evidencias diferentes: lectura exitosa, escritura denegada y exposición de puerto limitada. Ninguna acredita cifrado nativo ni integración LDAP. Declara explícitamente qué controles sí probaste y cuáles serían el siguiente despliegue de seguridad.

Por qué ocurre

Para cerrar, piensa en una aplicación de facturación: cliente por TLS, identidad de aplicación con roles mínimos, índices para consultas frecuentes, backups comprobados y cifrado de almacenamiento según requisitos. Redacta una tabla amenaza → control → comprobación → límite. No uses la expresión «base de datos segura» sin especificar contra qué amenaza y con qué evidencia. La capacidad de argumentar los límites es un resultado de aprendizaje tan importante como saber ejecutar mongosh.

Qué no permite concluir

Una lista de comprobación no reemplaza pruebas de penetración. La matriz final debe identificar para cada amenaza un control, una verificación y una limitación, evitando expresiones absolutas como «seguro».

Contraste resuelto con un segundo escenario

```
db.runCommand({connectionStatus:1})
```

Cierra con una matriz de amenazas y pruebas: no se da por demostrado LDAP ni cifrado nativo solo porque SCRAM funcione.

Ejercicio de transferencia

Proponer tres medidas obligatorias para producción.

Criterio de corrección de esta práctica

La corrección del ejercicio exige acreditar entregar matriz autenticado/no autenticado, read/write y riesgo. Añade el nombre exacto de la colección, del índice o de la cuenta y explica si el resultado es una operación permitida, una denegación deliberada o una conclusión de diseño no ejecutada.

Diario de comparación / conclusión

Qué esperaba	Qué observé	Qué puedo concluir